Squatch Signal

End-to-end working guide

Keyless lead intelligence: paste company websites, get scored, evidence-backed prospects, review and save them, export to CSV and draft outreach. Optional Supabase accounts keep everything permanently.

Section	What you will do
1. Overview	Understand what the app does and how data moves
2. Install and run	Get the app running locally (Windows, macOS, Linux)
3. Using the app	Every page, step by step
4. Supabase setup	Create the project, keys, tables and security
5. Data model	The three tables and what goes where
6. Verify end to end	A checklist proving discovery is saved
7. Deploy and update	Free hosting on Streamlit Community Cloud, plus how to ship changes
8. Schema report query	One query that prints your whole database layout
9. Troubleshooting	Exact errors, causes and fixes
10. Limits and responsible use	What the tool cannot do

1. Overview

Squatch Signal reads the public pages of company websites you provide (home, contact, about, careers, news) and produces a prospect record: clean company name, industry, headcount, ranked public emails, phone, LinkedIn and sentence-level buying signals such as hiring, funding, launches and expansion. Each prospect gets a transparent 0-100 fit score where every point has a stated reason.

Two ways to run it

Mode	Needs keys?	Where data lives	Best for
Guest	No	Browser session memory only (lost on refresh). Export CSV to keep.	Trying the app, demos
Account (Supabase)	Yes: URL and publishable key	Your Supabase Postgres, private to your login via row-level security	Real use, permanent history

How data flows

You do this	It is stored here
Discover prospects	Shown for review and logged to **discovered_leads** (signed-in users only)
Save N selected prospects to my leads	Copied into **leads**; duplicates (domain, then email, then company) are skipped
Add a Lead / Import CSV	**leads**
Save outreach draft	**outreach_drafts**

> **KEY IDEA. discovered_leads** is a history log: discovering the same URL twice adds rows twice. **leads** is your curated pipeline. Guests never write to Supabase, so guest discoveries will never appear in the dashboard.

2. Install and run

Requirements

- Python 3.12 (3.10+ should work)
- A modern browser
- Internet access (the app reads public websites)
- Optional: a free Supabase account for permanent storage

Windows (one click)

1. Unzip the project folder.
2. Double-click **run_app.bat**. It creates `.venv`, installs requirements and starts Streamlit.
3. Open `http://localhost:8501` and choose **Continue as guest** or sign in.

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

Windows manual

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m streamlit run app.py
```

> **NOTE.** If port 8501 is busy: `streamlit run app.py --server.port 8502`. If Windows reports "Application Control policy has blocked this file", install Python system-wide, recreate .venv from it, or deploy to Streamlit Cloud (Section 7).

Run the offline tests

```
python -m unittest -v      # 21 tests, about 8 seconds, no internet needed
```

3. Using the app

Use the sidebar to move between pages. The sidebar also shows who you are signed in as and has the sign-out button.

Sign in screen

- **Continue as guest**: instant, no account, data kept in this browser session only.
- **Sign in / Create account**: appears only when Supabase keys are configured. If email confirmation is ON, new users must click the emailed link before they can sign in.

Discover Leads (the main workflow)

1. Paste up to 10 company websites, one per line (for example `hubspot.com`).
2. Leave **Also read contact, about, careers and news pages** on for more emails, headcount and signals (adds a few seconds per company).
3. Leave the **Internet Archive fallback** on so bot-blocking sites still return a labelled, possibly outdated copy.
4. Click **Discover prospects**. Companies are processed in parallel and cached for one hour.
5. Review each prospect: score, signals, emails, phone, LinkedIn, source evidence and the score breakdown.
6. Untick prospects you do not want, then click **Save N selected prospect(s) to my leads**.
7. Optionally download the discovery results as CSV or clear the 1-hour cache.

> **NOTE.** Every discovery run is also written to **discovered_leads** automatically when you are signed in. If that write fails you will see a yellow banner starting "Could not store discovery history" with the database error.

Leads

Your saved pipeline: search, review, edit, delete and export to CSV. Scores shown are the fit score saved with each lead.

Import CSV

Upload a CSV; headers are matched flexibly (case and punctuation ignored). Accepted header names:

Field	Accepted headers
company	company, company_name, organization, account
domain	domain, website, company_domain
contact_name	contact_name, contact, person, full_name
contact_title	contact_title, title, job_title, role
industry	industry, sector
employees	employees, employee_count, company_size
revenue (USD millions)	estimated_revenue_usd_m, revenue, revenue_m, estimated_revenue
email / status	email, work_email, business_email / email_status, status
fit_score	fit_score, score, lead_score
signals / evidence	signals, growth_signals, reason_to_contact / evidence, source, source_notes

Duplicates already in your leads are skipped. Company name is required; email must be valid if present.

Add a Lead

A manual form for a single lead. The fit score can be calculated for you from the fields you fill in.

Outreach Drafts

Pick a lead, describe your value proposition in one line and generate a draft email. The draft uses the lead's recorded signal as the opener. Edit it, save it, and review every message yourself before sending anything.

Overview

Summary cards and charts for your saved leads: counts, average score, email coverage and industry mix.

4. Supabase setup (optional, for permanent accounts)

1. Create a free project at supabase.com and wait until it is ready.
2. **Authentication, Sign In / Providers**: keep **Email** enabled. For quick demos turn **Confirm email** off.
3. **Settings, API**: copy the **Project URL** and the **publishable (anon) key**. Never copy the secret or service_role key into this app.
4. In the project folder copy `.env.example` to `.env` and fill it in (below).
5. Open **SQL Editor**, paste the whole of `supabase_schema.sql` and run it. It is safe to re-run.
6. Run `python check_supabase.py`. Every line should say OK.
7. Restart the app and sign in with a real account.
8. Once deployed (Section 7), set **Authentication, URL Configuration, Site URL** to your live app address so confirmation emails do not point to localhost.

The .env file

```
SUPABASE_URL=https://abcdefghijklmnopqrst.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxx
```

> **SECURITY.** The app reads only **.env**, never .env.example. Do not commit .env or paste keys into chats and screenshots. If a secret or service_role key was ever exposed, rotate it in Supabase.

Why row-level security matters

Every table has row-level security (RLS) switched on and a policy that lets a signed-in user touch only rows whose **user_id** equals their own id. RLS on with no policy blocks everything, which is exactly what happened to **discovered_leads** before it received a policy. The Supabase Table Editor and SQL Editor run as an admin and bypass RLS, so you can see all users' rows there; the app cannot.

5. Data model

leads (your saved pipeline)

Column	Type	Notes
id	uuid	Primary key, auto-generated
user_id	uuid	Owner. References auth.users, cascade delete
company	text	Required
domain, industry, signals, evidence	text	Evidence includes the source URL
contact_name, contact_title	text	Legacy twins: contact, title (kept in sync)
employees	integer	Headcount estimate
estimated_revenue_usd_m	numeric	Legacy twin: revenue
fit_score	integer	CHECK 0-100. Legacy twin: score
email	text	Lower-cased
email_status	text	CHECK: Verified, Unverified or Missing (default Missing)
created_at, updated_at	timestamptz	Default now()

discovered_leads (discovery history)

Column	Type	Notes
id	bigint identity	Primary key, auto-generated
user_id	uuid	Owner, defaults to auth.uid()
company, domain, industry	text	From the discovery engine
email, email_status	text	Best-ranked email and its check result
all_emails	text	Comma-separated list of other emails found
signals, evidence	text	Buying signals and source text
source_url, page_title, description, content_type	text	Where the data came from
discovered_at, created_at	timestamptz	Default now()

outreach_drafts

Column	Type	Notes
id, user_id	uuid	Owner via user_id
lead_id	uuid	Optional link to leads; set null if the lead is deleted
subject, body	text	body is required
status	text	CHECK: draft, reviewed, sent or archived
created_at, updated_at	timestamptz	Default now()

6. Verify end to end

Work through this checklist once after setup. If a step fails, jump to Section 9.

#	Check	Expected result
1	Run `python check_supabase.py`	All OK, final line "All checks passed."
2	Start the app and sign in (not guest)	Sidebar shows your email, not "Guest workspace"
3	Discover Leads with `hubspot.com`	A prospect card appears with score and evidence; no yellow warning
4	Supabase, Table Editor, **discovered_leads**	New row(s) with your user_id and the source_url
5	Click Save selected prospects	Green "Saved N lead(s)"; app jumps to Leads
6	Table Editor, **leads**	The saved lead is there. Saving it again says "Skipped 1 duplicate"
7	Outreach Drafts, generate and save	Row appears in **outreach_drafts**

Quick SQL to see what is stored

```
select count(*) as discovered from public.discovered_leads;
select count(*) as leads      from public.leads;

select company, domain, email, source_url, discovered_at
from public.discovered_leads
order by discovered_at desc
limit 20;
```

7. Deploy on Streamlit Community Cloud and keep it updated

> **STATUS.** Repository: github.com/Vijaylal8055/squatch-signal (public, branch main). Deployed and verified: sign-in works and every discovery run is stored in the Supabase table discovered_leads.

What the free tier means (can change; check the Streamlit docs)

- Free, needs a GitHub account; unlimited public apps, one private app, no custom domain, address ends in .streamlit.app.
- Roughly 1 GB memory, shared by all free apps.
- Apps with no traffic for about 12 hours go to sleep. The next visitor clicks a wake-up button. The 1-hour discovery cache and guest data are lost on sleep.
- Discovery runs from Streamlit's shared servers, so a few sites may answer HTTP 403; the Internet Archive fallback usually still returns a labelled copy.
- Free Supabase projects can pause when idle; restore the project from the Supabase dashboard if sign-in suddenly fails.

First deployment

1. Create an empty **public** repo on GitHub named `squatch-signal` (no README, no .gitignore).
2. In the project folder run the commands below. Read the `git status` list: **.env must not appear**.
3. At share.streamlit.io choose **Create app, Deploy a public app from GitHub**. Repository `Vijaylal8055/squatch-signal`, branch `main`, main file `app.py`.
4. Open **Advanced settings**: select Python 3.12 and paste the Secrets below (publishable key only).
5. Click **Deploy**. The first build takes a couple of minutes.
6. In Supabase set **Site URL** to your `https://subdomain.streamlit.app` address.
7. Test: sign in, discover a site, then look for new rows in Supabase **discovered_leads**.

```
git init
git add .
git status
git commit -m "Squatch Signal"
git branch -M main
git remote add origin https://github.com/<you>/squatch-signal.git
git push -u origin main
```

```
SUPABASE_URL = "https://abcdefghijklmnopqrst.supabase.co"
SUPABASE_PUBLISHABLE_KEY = "sb_publishable_xxxxxxxxxxxxxxxxxxxx"
```

Updating the app later

```
git add .
git commit -m "describe your change"
git push
```

Streamlit Cloud redeploys automatically after each push to main. Secrets are changed in the app's Settings, Secrets page and need no push.

> **SECURITY.** GitHub no longer accepts account passwords for pushes. Sign in through the browser prompt, run `gh auth login`, or use a Personal Access Token. If .env ever reaches a public repo, delete it and rotate the key in Supabase.

8. Schema report query

Paste this into Supabase SQL Editor to print columns, RLS status, policies, constraints, indexes, triggers and row counts as one JSON result. Use it whenever you need to see or share the exact layout of your database.

```
select jsonb_pretty(jsonb_build_object(
  'columns', (select jsonb_agg(jsonb_build_object(
      'table', table_name, 'column', column_name, 'type', data_type,
      'nullable', is_nullable, 'default', column_default)
      order by table_name, ordinal_position)
    from information_schema.columns where table_schema = 'public'),
  'rls_status', (select jsonb_agg(jsonb_build_object(
      'table', c.relname, 'rls_enabled', c.relrowsecurity,
      'rls_forced', c.relforcerowsecurity))
    from pg_class c join pg_namespace n on n.oid = c.relnamespace
    where n.nspname = 'public' and c.relkind = 'r'),
  'policies', (select jsonb_agg(jsonb_build_object(
      'table', tablename, 'policy', policyname, 'command', cmd,
      'roles', roles, 'using', qual, 'with_check', with_check))
    from pg_policies where schemaname = 'public'),
  'constraints', (select jsonb_agg(jsonb_build_object(
      'table', conrelid::regclass::text, 'name', conname,
      'definition', pg_get_constraintdef(oid)))
    from pg_constraint where connamespace = 'public'::regnamespace),
  'indexes', (select jsonb_agg(jsonb_build_object(
      'table', tablename, 'index', indexname, 'definition', indexdef))
    from pg_indexes where schemaname = 'public'),
  'triggers', (select jsonb_agg(jsonb_build_object(
      'table', event_object_table, 'trigger', trigger_name,
      'event', event_manipulation))
    from information_schema.triggers where trigger_schema = 'public'),
  'leads_total_rows', (select count(*) from public.leads),
  'discovered_total_rows', (select count(*) from public.discovered_leads),
  'drafts_total_rows', (select count(*) from public.outreach_drafts),
  'leads_rows_per_user', (select jsonb_agg(jsonb_build_object(
      'user_id', user_id, 'rows', n))
    from (select user_id, count(*) n from public.leads group by user_id) t)
)) as schema_report;
```

> **NOTE.** A column that shows `default: null` may still be an identity column (auto-numbered). That is why running `alter column id add generated ... as identity` returns error 55000 on discovered_leads.

9. Troubleshooting

Symptom or error	Cause	Fix
Discovered rows never reach Supabase	Guest mode, missing .env values, or no RLS policy on discovered_leads	Sign in with a real account; run check_supabase.py; re-run supabase_schema.sql
Yellow banner: Could not store discovery history	The database rejected the insert; the message names why	Read the message: 42501 means RLS or not signed in; 42703 means a missing column. Re-run the schema
42710: policy "own leads" already exists	create policy run without dropping it first	Use supabase_schema.sql (it drops first) or: drop policy if exists "own leads" on public.leads;
55000: column "id" is already an identity column	The column is already auto-numbered	Skip that statement; it is already correct. The whole script rolls back on error, so run the remaining statements
42501: new row violates row-level security policy	Not signed in, or table has RLS but no policy, or user_id mismatch	Sign in; confirm a policy exists in the schema report; re-run the schema
23514: violates check constraint	email_status not Verified/Unverified/Missing; fit_score outside 0-100; draft status invalid	Fix the value; the app already normalises these for normal use
Account created but cannot sign in	Email confirmation is on	Click the emailed link or turn off Confirm email in Supabase
Authentication failed / key rejected	Wrong key, wrong project, or a secret key was used	Use the publishable or anon key from the same project; run check_supabase.py
Project unreachable	Free projects pause when idle	Open the Supabase dashboard and restore the project
Old discovered_leads rows invisible in the app	Rows have null user_id so RLS hides them	Assign them: update public.discovered_leads set user_id = '' where user_id is null;
A site returns HTTP 403	It blocks automated clients	Expected: the app reports it and never bypasses blocks; the archive fallback may supply a labelled copy
Deployed app shows a wake-up page	It slept after about 12 idle hours (free tier)	Click the wake-up button and wait for the restart
Confirmation email opens localhost	Supabase Site URL is still the default	Set Site URL to your .streamlit.app address (Authentication, URL Configuration)
Works locally, fails on Streamlit Cloud	Secrets not valid TOML, or wrong Python version	Check Manage app, Logs; quote the values; select Python 3.12
Port 8501 busy	Another Streamlit is running	streamlit run app.py --server.port 8502
Application Control policy blocked a file	Windows AppLocker/WDAC	Install Python system-wide, recreate .venv, or deploy to Streamlit Cloud

10. Limits and responsible use

- Reads server-rendered HTML. JavaScript-only sites expose little, and bot-blocking sites return an error or a labelled archived copy.
- Industry, signals and headcount are keyword heuristics, not AI predictions. Treat them as leads for human review.
- Email checks prove the domain can receive mail (MX), not that a mailbox exists.
- It finds what a company publishes about itself; it is not a people database.
- Honours robots.txt, visits only URLs you provide, blocks private and local addresses, and never bypasses logins, CAPTCHAs or rate limits.
- Follow each site's terms, privacy law (such as GDPR) and anti-spam rules (such as CAN-SPAM). A human should review every outreach message before it is sent.

Files in this project

File	Purpose
app.py	Streamlit UI and database calls
lead_discovery.py	Keyless discovery engine
local_db.py	Guest in-memory workspace
supabase_schema.sql	Tables, RLS policies, indexes, checks (safe to re-run)
check_supabase.py	Read-only configuration checker for all three tables
test_lead_discovery.py, test_app.py	Offline tests (21)
requirements.txt, run_app.bat, .env.example	Dependencies, Windows launcher, config template